

II.9 Extrapolation Methods

The following method of approximation may or may not be new, but as I believe it to be of practical importance . . .

(S.A. Corey 1906)

The h^2 -extrapolation was discovered by a hint from theory followed by arithmetical experiments, which gave pleasing results.

(L.F. Richardson 1927)

Extrapolation constitutes a powerful means . . .

(R. Bulirsch & J. Stoer 1966)

Extrapolation does not appear to be a particularly effective way . . . , our tests raise the question as to whether there is any point to pursuing it as a separate method.

(L.F. Shampine & L.S. Baca 1986)

Definition of the Method

Let $y' = f(x, y)$, $y(x_0) = y_0$ be a given differential system and $H > 0$ a basic step size. We choose a sequence of positive integers

$$n_1 < n_2 < n_3 < \dots \quad (9.1)$$

and define the corresponding step sizes $h_1 > h_2 > h_3 > \dots$ by $h_i = H/n_i$. We then choose a numerical method of order p and compute the numerical results of our initial value problem by performing n_i steps with step size h_i to obtain

$$y_{h_i}(x_0 + H) =: T_{i,1} \quad (9.2)$$

(the letter “ T ” stands historically for “trapezoidal rule”). We then eliminate as many terms as possible from the asymptotic expansion (8.10) by computing the interpolation polynomial

$$p(h) = \hat{y} - e_p h^p - e_{p+1} h^{p+1} - \dots - e_{p+k-2} h^{p+k-2} \quad (9.3)$$

such that

$$p(h_i) = T_{i,1} \quad i = j, j-1, \dots, j-k+1. \quad (9.4)$$

Finally we “*extrapolate to the limit*” $h \rightarrow 0$ and use

$$p(0) = \hat{y} =: T_{j,k}$$

as numerical result. Conditions (9.4) consist of k linear equations for the k unknowns $\hat{y}, e_p, \dots, e_{p+k-2}$.

Example. For $k = 2$, $n_1 = 1$, $n_2 = 2$ the above definition is identical to Richardson’s extrapolation discussed in Section II.4.

Theorem 9.1. *The value $T_{j,k}$ represents a numerical method of order $p+k-1$.*

Proof. We compare (9.4) and (9.3) with the asymptotic expansion (8.10) which we write in the form (with $N = p+k-1$)

$$T_{i,1} = y(x_0+H) - e_p(x_0+H)h_i^p - \dots - e_{p+k-2}(x_0+H)h_i^{p+k-2} - \Delta_i, \quad (9.4')$$

where

$$\Delta_i = e_{p+k-1}(x_0+H)h_i^{p+k-1} + E_{h_i}(x_0+H)h_i^{p+k} = \mathcal{O}(H^{p+k})$$

because $e_{p+k-1}(x_0) = 0$ and $h_i \leq H$. This is a linear system for the unknowns $y(x_0+H)$, $H^p e_p(x_0+H)$, $\dots$, $H^{p+k-2} e_{p+k-2}(x_0+H)$ with the Vandermonde-like matrix

$$A = \begin{pmatrix} 1 & \frac{1}{n_j^p} & \dots & \frac{1}{n_j^{p+k-2}} \\ \vdots & \vdots & & \vdots \\ 1 & \frac{1}{n_{j-k+1}^p} & \dots & \frac{1}{n_{j-k+1}^{p+k-2}} \end{pmatrix}.$$

It is the same as (9.4), just with the right-hand side perturbed by the $\mathcal{O}(H^{p+k})$ -terms Δ_i . The matrix A is invertible (see Exercise 6). Therefore by subtraction we obtain

$$|y(x_0+H) - \hat{y}| \leq \|A^{-1}\|_\infty \cdot \max |\Delta_i| = \mathcal{O}(H^{p+k}). \quad \square$$

Remark. The case $p=1$ (as well as $p=2$ with expansions in h^2) can also be treated by interpreting the difference $y(x_0+H) - \hat{y}$ as an interpolation error (see (9.21)).

A great advantage of the method is that it provides a complete table of numerical results

$$\begin{array}{ccccccc} T_{11} & & & & & & \\ T_{21} & T_{22} & & & & & \\ T_{31} & T_{32} & T_{33} & & & & \\ T_{41} & T_{42} & T_{43} & T_{44} & & & \\ \dots & \dots & \dots & \dots & \dots & & \end{array} \quad (9.5)$$

which form a sequence of embedded methods and allow easy estimates of the local error and strategies for variable order. Several step-number sequences are in use for (9.1):

The “Romberg sequence” (Romberg 1955):

$$1, 2, 4, 8, 16, 32, 64, 128, 256, 512, \dots \quad (9.6)$$

The “*Bulirsch sequence*” (see also Romberg 1955):

$$1, 2, 3, 4, 6, 8, 12, 16, 24, 32, \dots \quad (9.7)$$

alternating powers of 2 with 1.5 times 2^k . This sequence needs fewer function evaluations for higher orders than the previous one and became prominent through the success of the “Gragg-Bulirsch-Stoer algorithm” (Bulirsch & Stoer 1966).

The above sequences have the property that for integration problems $y' = f(x)$ many function values can be saved and re-used for smaller h_i . Further, $\liminf(n_{i+1}/n_i)$ remains bounded away from 1 (“Toeplitz condition”) which allows convergence proofs for $j = k \rightarrow \infty$ (Bauer, Rutishauser & Stiefel 1963). However, if we work with differential equations and with fixed or bounded order, the most economic sequence is the “*harmonic sequence*” (Deuffhard 1983)

$$1, 2, 3, 4, 5, 6, 7, 8, 9, 10, \dots \quad (9.8)$$

The Aitken - Neville Algorithm

For the case $p = 1$, (9.3) and (9.4) become a classical interpolation problem and we can compute the values of $T_{j,k}$ economically by the use of classical methods. Since we need only the values of the interpolation polynomials at the point $h = 0$, the most economical algorithm is that of “Aitken - Neville” (Aitken 1932, Neville 1934, based on ideas of Jordan 1928) which leads to

$$T_{j,k+1} = T_{j,k} + \frac{T_{j,k} - T_{j-1,k}}{(n_j/n_{j-k}) - 1}. \quad (9.9)$$

If the basic method used is *symmetric*, we know that the underlying asymptotic expansion is in powers of h^2 (Theorem 8.9), and each extrapolation eliminates *two* powers of h . We may thus simply replace in (9.3) h by h^2 and for $p = 2$ (i.e., $q = 1$ in (8.18)) also use the Aitken - Neville algorithm with this modification. This leads to

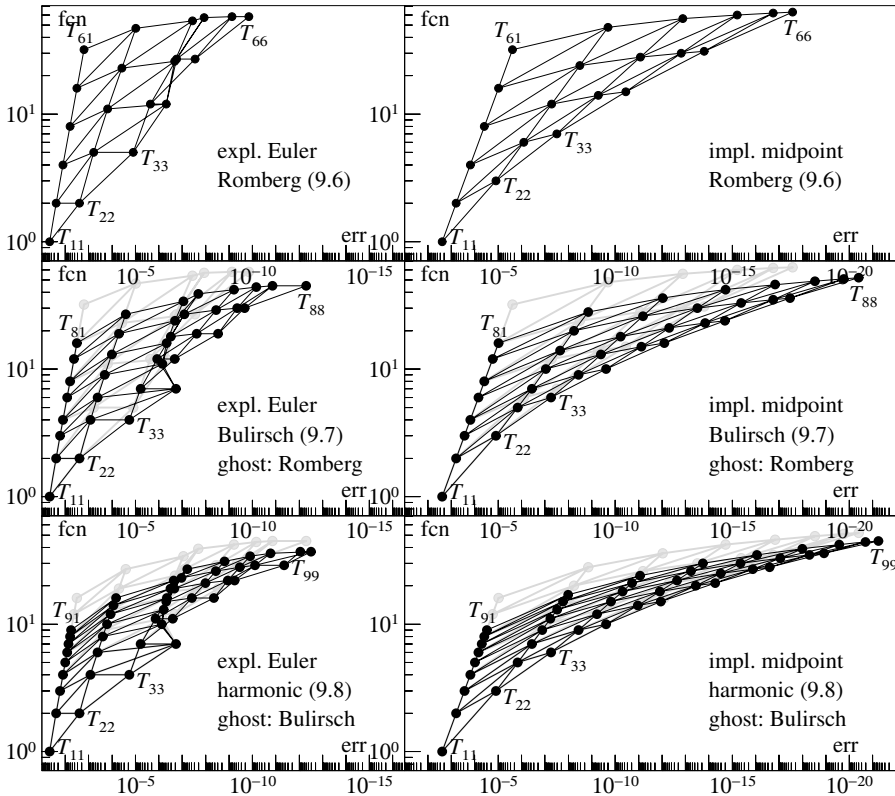
$$T_{j,k+1} = T_{j,k} + \frac{T_{j,k} - T_{j-1,k}}{(n_j/n_{j-k})^2 - 1} \quad (9.10)$$

instead of (9.9).

Numerical example. We solve the problem

$$y' = (-y \sin x + 2 \tan x)y, \quad y(\pi/6) = 2/\sqrt{3} \quad (9.11)$$

with true solution $y(x) = 1/\cos x$ and basic step size $H = 0.2$ by Euler’s method. Fig. 9.1 represents, for each of the entries $T_{j,k}$ of the extrapolation tableau, the *numerical work* $(1 + n_j - 1 + n_{j-1} - 1 + \dots + n_{j-k+1} - 1)$ compared to the *precision* $(|T_{j,k} - y(x_0 + H)|)$ in double logarithmic scale. The first picture is for the Romberg sequence (9.6), the second for the Bulirsch sequence (9.7), and the last

Fig. 9.1. h -extrap. expl. EulerFig. 9.2. h^2 -extrap. impl. midpoint

for the harmonic sequence (9.8). In pictures 2 and 3 the results of the foregoing graphics are repeated as a shaded “ghost” (. . . of Canterbury) in order to demonstrate how the results are better than those for the predecessor. Nobody is perfect, however. The “best” method in these comparisons, the harmonic sequence, suffers for high orders from a strong influence of rounding errors (see Exercise 5 below; the computations of Fig. 9.1, 9.2 and 9.4 have been made in quadruple precision).

The analogous results for the symmetric implicit mid-point rule (7.4) are presented in Fig. 9.2. Although implicit, this method is easy to implement for this particular example. We again use the same basic step size $H = 0.2$ as above and the same step-number sequences (9.6), (9.7), (9.8). Here, the “numerical work” ($n_j + n_{j-1} + \dots + n_{j-k+1}$) represents *implicit* stages and therefore can not be compared to the values of the explicit method. The precisions, however, show a drastic improvement.

Rational Extrapolation. Many authors in the sixties claimed that it is better to use rational functions instead of polynomials in (9.3). In this case the formula (9.9)

must be replaced by (Bulirsch & Stoer 1964)

$$T_{j,k+1} = T_{j,k} + \frac{T_{j,k} - T_{j-1,k}}{\left(\frac{n_j}{n_{j-k}}\right) \left(1 - \frac{T_{j,k} - T_{j-1,k}}{T_{j,k} - T_{j-1,k-1}}\right) - 1} \quad (9.12)$$

where

$$T_{j,0} = 0.$$

For systems of differential equations the division of vectors is to be understood componentwise.

Later numerical experiments (Deuffhard 1983) showed that rational extrapolation is nearly never more advantageous than polynomial extrapolation.

The Gragg or GBS Method

Since it is fully explicit GRAGG's algorithm is so ideally suited as a basis for RICHARDSON extrapolation that no other symmetric two-step algorithm can compete with it. (H.J. Stetter 1970)

Here we can not do better than quote from Stetter (1970): "Expansions in powers of h^2 are extremely important for an efficient application of Richardson extrapolation. Therefore it was a great achievement when Gragg proved in 1963 that the quantity $S_h(x)$ produced by the algorithm ($x = x_0 + 2nh$, $x_i = x_0 + ih$)

$$y_1 = y_0 + hf(x_0, y_0) \quad (9.13a)$$

$$y_{i+1} = y_{i-1} + 2hf(x_i, y_i) \quad i = 1, 2, \dots, 2n \quad (9.13b)$$

$$S_h(x) = \frac{1}{4} (y_{2n-1} + 2y_{2n} + y_{2n+1}) \quad (9.13c)$$

possesses an asymptotic expansion in even powers of h and has satisfactory stability properties. This led to the construction of the very powerful G(ragg)-B(ulirsch)-S(toer)-extrapolation algorithm . . .".

Gragg's *proof* of this property was very long and complicated and it was again "a great achievement" that Stetter had the elegant idea of interpreting (9.13b) as a *one-step* algorithm by rewriting (9.13) in terms of odd and even indices: for this purpose we define

$$\begin{aligned} h^* &= 2h, & x_k^* &= x_0 + kh^*, & u_0 &= v_0 = y_0, \\ u_k &= y_{2k}, & v_k &= y_{2k+1} - hf(x_{2k}, y_{2k}) = \frac{1}{2} (y_{2k+1} + y_{2k-1}). \end{aligned} \quad (9.14)$$

Then the method (9.13) can be rewritten as (see Fig. 9.3)

$$\begin{pmatrix} u_{k+1} \\ v_{k+1} \end{pmatrix} = \begin{pmatrix} u_k \\ v_k \end{pmatrix} + h^* \begin{pmatrix} f\left(x_k^* + \frac{h^*}{2}, v_k + \frac{h^*}{2} f(x_k^*, u_k)\right) \\ \frac{1}{2} \left(f(x_k^* + h^*, u_{k+1}) + f(x_k^*, u_k)\right) \end{pmatrix}. \quad (9.15)$$

approximately the same amount (see Fig. IV.2.3 of Vol. II). Further, it has the advantage of evaluating the function f at the end of the basic step.

Theorem 9.2. *Let $f(x, y) \in \mathcal{C}^{2\ell+2}$, then the numerical solution defined in (9.13) possesses for $x = x_0 + 2nh$ an asymptotic expansion of the form*

$$y(x) - S_h(x) = \sum_{j=1}^{\ell} e_{2j}(x)h^{2j} + h^{2\ell+2}C(x, h) \quad (9.19)$$

with $e_{2j}(x_0) = 0$ and $C(x, h)$ bounded for $x_0 \leq x \leq \bar{x}$ and $0 \leq h \leq h_0$.

Proof. By adding (9.17a) and (9.17b) and using $h^* = 2h$ we obtain (9.19) with $e_{2j}(x) = (a_{2j}(x) + b_{2j}(x))2^{2j-1}$. $\square$

This method can thus be used for Richardson extrapolation in the same way as symmetric methods above: we choose a step-number sequence, with the condition that the n_j are even, i.e.,

$$2, 4, 8, 16, 32, 64, 128, 256, \dots \quad (9.6')$$

$$2, 4, 6, 8, 12, 16, 24, 32, 48, \dots \quad (9.7')$$

$$2, 4, 6, 8, 10, 12, 14, 16, 18, \dots \quad (9.8')$$

set

$$T_{i,1} := S_{h_i}(x_0 + H)$$

and compute the extrapolated expressions $T_{i,j}$, based on the h^2 -expansion, by the Aitken-Neville formula (9.10).

Numerical example. Fig. 9.4 represents the numerical results of this algorithm applied to Example (9.11) with step size $H = 0.2$. The step size sequences are Romberg (9.6') (above), Bulirsch (9.7') (middle), and harmonic (9.8') (below). The algorithm *with* smoothing step (numerical work = $1 + n_j + n_{j-1} + \dots + n_{j-k+1}$) is represented *left*, the results *without* smoothing step (numerical work = $1 + n_j - 1 + n_{j-1} - 1 + \dots + n_{j-k+1} - 1$) are on the *right*.

The results are nearly identical to those for the implicit midpoint rule (Fig. 9.2), but much more valuable, since here the method is explicit. In the pictures on the left the values for extrapolated Euler (from Fig. 9.1) are repeated as a “ghost” and demonstrate clearly the importance of the h^2 -expansion, especially in the diagonal T_{kk} for large values of k . The ghost in the pictures on the right are the values *with* smoothing from the left; the differences are seen to be tiny.

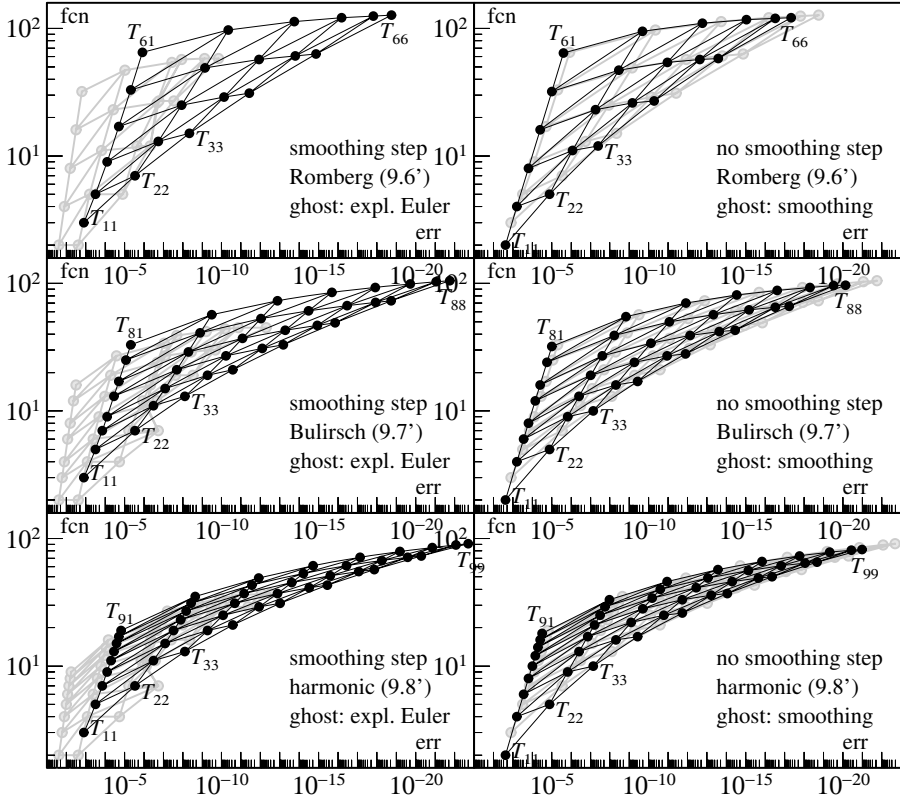


Fig. 9.4. Precision of h^2 -extrapolated Gragg method for Example (9.11)

Asymptotic Expansion for Odd Indices

For completeness, we still want to derive the existence of an h^2 expansion for y_{2k+1} from (9.17b), although this is of no practical importance for the numerical algorithm described above.

Theorem 9.3 (Gragg 1964). *For $x = x_0 + (2k+1)h$ we have*

$$y(x) - y_h(x) = \sum_{j=1}^{\ell} \hat{b}_{2j}(x) h^{2j} + h^{2\ell+2} \hat{B}(x, h) \quad (9.20)$$

where the coefficients $\hat{b}_{2j}(x)$ are in general different from those for even indices and $\hat{b}_{2j}(x_0) \neq 0$.

Proof. y_{2k+1} can be computed (see Fig. 9.3) either from v_k by a forward step or from v_{k+1} by a backward step. For the sake of symmetry, we take the mean of

both expressions and write

$$y_{2k+1} = \frac{1}{2}(v_k + v_{k+1}) + \frac{h}{2}(f(x_k^*, u_k) - f(x_{k+1}^*, u_{k+1})).$$

We now subtract the exact solution and obtain

$$\begin{aligned} 2(y_h(x) - y(x)) &= v_{2h}(x-h) - y(x-h) \\ &\quad + v_{2h}(x+h) - y(x+h) + y(x-h) - 2y(x) + y(x+h) \\ &\quad + h\left(f(x-h, u_{2h}(x-h)) - f(x+h, u_{2h}(x+h))\right). \end{aligned}$$

Due to the symmetry of $u_{2h}(x)$ ($u_{2h}(\xi) = u_{-2h}(\xi)$) and of $v_{2h}(x)$ the whole expression becomes symmetric in h . Thus the asymptotic expansion for y_{2k+1} contains no odd powers of h . $\square$

Both expressions, for even and for odd indices, can still be combined into a single formula (see Exercise 2).

Existence of Explicit RK Methods of Arbitrary Order

Each of the expressions $T_{j,k}$ clearly represents an explicit RK-method (see Exercise 1). If we apply the well-known error formula for polynomial interpolation (see e.g., Abramowitz & Stegun 1964, formula 25.2.27) to (9.19), we obtain

$$y(x_0 + H) - T_{j,k} = \frac{(-1)^k}{n_j^2 \cdot \dots \cdot n_{j-k+1}^2} e_{2k}(x_0 + H) H^{2k} + \mathcal{O}(H^{2k+2}). \quad (9.21)$$

Since $e_k(x_0) = 0$, we have

$$y(x_0 + H) - T_{j,k} = \frac{(-1)^k}{n_j^2 \cdot \dots \cdot n_{j-k+1}^2} e'_{2k}(x_0) H^{2k+1} + \mathcal{O}(H^{2k+2}). \quad (9.22)$$

This shows that $T_{j,k}$ represents an explicit Runge-Kutta method of order $2k$. As an application of this result we have:

Theorem 9.4 (Gragg 1964). *For p even, there exists an explicit RK-method of order p with $s = p^2/4 + 1$ stages.*

Proof. This result is obtained by counting the number of necessary function evaluations of the GBS-algorithm using the harmonic sequence and without the final smoothing step. $\square$

Remark. The extrapolated Euler method leads to explicit Runge-Kutta methods with $s = p(p-1)/2 + 1$ stages. This shows once again the importance of the h^2 expansion.

Order and Step Size Control

Extrapolation methods have the advantage that in addition to the step size also the order (i.e., number of columns) can be changed at each step. Because of this double freedom, the *practical implementation* in an optimal way is more complicated than for fixed-order RK-methods. The first codes were developed by Bulirsch & Stoer (1966) and their students. Very successful extrapolation codes due to P. Deuffhard and his collaborators are described in Deuffhard (code DIFEX1, 1983).

The choice of the *step size* can be performed in exactly the same way as for fixed-order embedded methods (see Section II.4). If the first k lines of the extrapolation tableau are computed, we have T_{kk} as the highest-order approximation (of order $2k$ by (9.22)) and in addition $T_{k,k-1}$ of order $2k-2$. It is therefore natural to use the expression

$$err_k = \|T_{k,k-1} - T_{k,k}\| \quad (9.23)$$

for step size control. The norm is the same as in (4.11). As in (4.12) we get for the optimal step size the formula

$$H_k = H \cdot 0.94 \cdot (0.65/err_k)^{1/(2k-1)} \quad (9.24)$$

where this time we have chosen a safety factor depending partly on the order.

For the choice of an *optimal order* we need a measure of work, which allows us to compare different methods. The work for computing T_{kk} can be measured by the number A_k of function evaluations. For the GBS-algorithm it is given recursively by

$$\begin{aligned} A_1 &= n_1 + 1 \\ A_k &= A_{k-1} + n_k. \end{aligned} \quad (9.25)$$

However, a large number of function evaluations can be compensated by a large step size H_k , given by (9.24). We therefore consider

$$W_k = \frac{A_k}{H_k}, \quad (9.26)$$

the *work per unit step*, as a measure of work. The idea is now to choose the order (i.e., the index k) in such a way that W_k is minimized.

Let us describe the *combined order and step size control* in some more detail. We assume that at some point of integration the step size H and the index k ($k > 2$) are proposed. The step is then realized in the following way: we first compute $k-1$ lines of the extrapolation tableau and also the values H_{k-2} , W_{k-2} , err_{k-1} , H_{k-1} , W_{k-1} .

a) *Convergence in line $k-1$* . If $err_{k-1} \leq 1$, we accept $T_{k-1,k-1}$ as numerical solution and continue the integration with the new proposed quantities

$$k_{\text{new}} = \begin{cases} k & \text{if } W_{k-1} < 0.9 \cdot W_{k-2} \\ k-1 & \text{else} \end{cases} \quad (9.27)$$

$$H_{\text{new}} = \begin{cases} H_{k_{\text{new}}} & \text{if } k_{\text{new}} \leq k-1 \\ H_{k-1}(A_k/A_{k-1}) & \text{if } k_{\text{new}} = k. \end{cases}$$

In (9.27), the only non-trivial formula is the choice of the step size H_{new} in the case of an order-increase $k_{\text{new}} = k$. In this case we want to avoid the computation of err_k , so that H_k and W_k are unknown. However, since our k is assumed to be close to the optimal value, we have $W_k \approx W_{k-1}$ which leads to the proposed step size increase.

b) *Convergence monitor*. If $err_{k-1} > 1$, we first decide whether we may expect convergence at least in line $k+1$. It follows from (9.22) that, asymptotically,

$$\|T_{k,k-2} - T_{k,k-1}\| \approx \left(\frac{n_2}{n_k}\right)^2 err_{k-1} \quad (9.28)$$

with err_{k-1} given by (9.23). Unfortunately, err_k cannot be compared with (9.28), since different factors (depending on the differential equation to be solved) are involved in the asymptotic formula (cf. (9.22)). If we nevertheless assume that err_k is $(n_2/n_1)^2$ times smaller than (9.28) we obtain $err_k \approx (n_1/n_k)^2 err_{k-1}$. We therefore already reject the step at this point, if

$$err_{k-1} > \left(\frac{n_{k+1}n_k}{n_1n_1}\right)^2 \quad (9.29)$$

and restart with $k_{\text{new}} \leq k-1$ and H_{new} according to (9.27). If the contrary of (9.29) holds, we compute the next line of the extrapolation tableau, i.e., $T_{k,k}$, err_k , H_k and W_k .

c) *Convergence in line k* . If $err_k \leq 1$, we accept T_{kk} as numerical solution and continue the integration with the new proposed values

$$k_{\text{new}} = \begin{cases} k-1 & \text{if } W_{k-1} < 0.9 \cdot W_k \\ k+1 & \text{if } W_k < 0.9 \cdot W_{k-1} \\ k & \text{in all other cases} \end{cases} \quad (9.30)$$

$$H_{\text{new}} = \begin{cases} H_{k_{\text{new}}} & \text{if } k_{\text{new}} \leq k \\ H_k(A_{k+1}/A_k) & \text{if } k_{\text{new}} = k+1. \end{cases}$$

d) *Second convergence monitor*. If $err_k > 1$, we check, as in (b), the relation

$$err_k > \left(\frac{n_{k+1}}{n_1}\right)^2. \quad (9.31)$$

If (9.31) is satisfied, the step is rejected and we restart with $k_{\text{new}} \leq k$ and H_{new} of (9.30). Otherwise we continue.

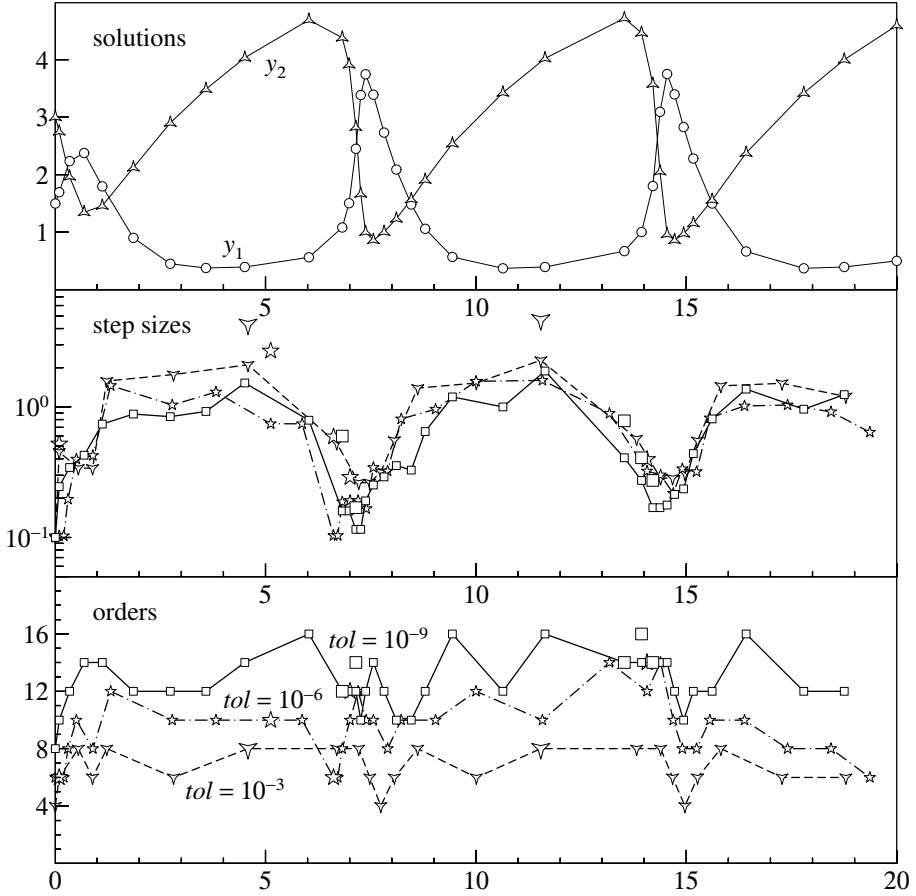


Fig. 9.5. Solution, step size and order variation obtained by ODEX

e) *Hope for convergence in line $k+1$.* We compute err_{k+1} , H_{k+1} and W_{k+1} . If $err_{k+1} \leq 1$, we accept $T_{k+1,k+1}$ as numerical solution and continue the integration with the new proposed order

$$\begin{aligned}
 k_{\text{new}} &:= k \\
 \text{if } (W_{k-1} < 0.9 \cdot W_k) & \quad k_{\text{new}} := k-1 \\
 \text{if } (W_{k+1} < 0.9 \cdot W_{k_{\text{new}}}) & \quad k_{\text{new}} := k+1.
 \end{aligned} \tag{9.32}$$

If $err_{k+1} > 1$ the step is rejected and we restart with $k_{\text{new}} \leq k$ and H_{new} of (9.24).

The following slight modifications of the above algorithm are recommended:

i) Storage considerations lead to a limitation of the number of columns of the extrapolation tableau, say by k_{max} (e.g., $k_{\text{max}} = 9$). For the proposed index k_{new} we require $2 \leq k_{\text{new}} \leq k_{\text{max}} - 1$. This allows us to activate (e) at each step.

ii) After a step-rejection the step size and the order may not be increased.

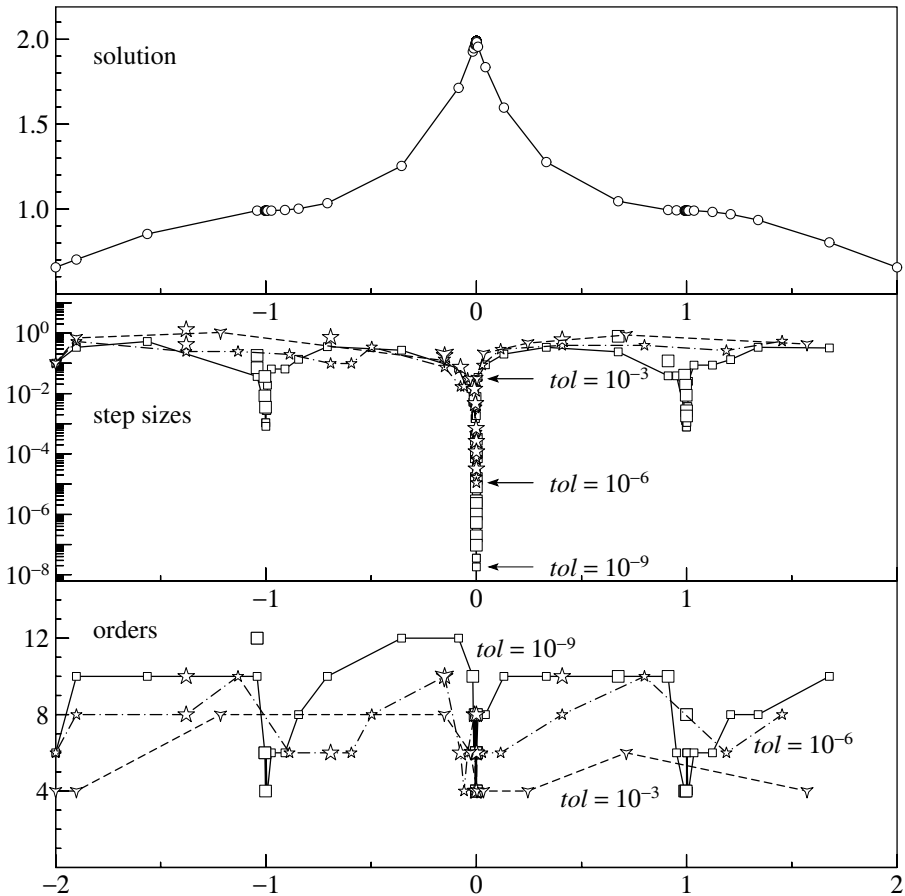


Fig. 9.6. Solution, step size and order variation obtained by ODEX at the discontinuous example (9.33)

Numerical study of the combined step size and order control. We show in the following examples how the step size and the order vary for the above algorithm. For this purpose we have written the FORTRAN-subroutine ODEX (see Appendix).

As a first example we again take the *Brusselator* (cf. Section II.4). As in Fig. 4.1, the first picture of Fig. 9.5 shows the two components of the solution (obtained with $Atol = Rtol = 10^{-9}$). In the remaining two pictures we have plotted the step sizes and orders for the three tolerances 10^{-3} (broken line), 10^{-6} (dashes and dots) and 10^{-9} (solid line). One can easily observe that the extrapolation code automatically chooses a suitable order (depending essentially on Tol). Step-rejections are indicated by larger symbols.

We next study the behaviour of the order control near discontinuities. In the example

$$y' = -\text{sign}(x) |1 - |x|| y^2, \quad y(-2) = 2/3, \quad -2 \leq x \leq 2 \quad (9.33)$$

we have a discontinuity in the first derivative of $y(x)$ at $x = 0$ and two discontinuities in the second derivative (at $x = \pm 1$). The numerical results are shown in Fig. 9.6 for three tolerances. In all cases the error at the endpoint is about $10 \cdot \text{Tot}$. The discontinuities at $x = \pm 1$ are not recognized in the computations with $\text{Tot} = 10^{-3}$ and $\text{Tot} = 10^{-6}$. Whenever a discontinuity is detected, the order drops to 4 (lowest possible) in its neighbourhood, so that these points are passed rather efficiently.

Dense Output for the GBS Method

Extrapolation methods are methods best suited for high precision which typically take very large (basic) step sizes during integration. The reasons for the need of a dense output formula (discussed in Section II.6) are therefore particularly important here. First attempts to provide extrapolation methods with a dense output are due to Lindberg (1972) for the implicit trapezoidal rule, and to Shampine, Baca & Bauer (1983) who constructed a 3rd order dense output for the GBS method. We present here the approach of Hairer & Ostermann (1990) (see also Simonsen 1990).

It turned out that the existence of high order dense output is only possible if the step number sequence satisfies some restrictions such as

$$n_{j+1} - n_j = 0 \pmod{4} \quad \text{for } j = 1, 2, 3, \dots \quad (9.34)$$

which, for example, is fulfilled by the sequence

$$\{2, 6, 10, 14, 18, 22, 26, 30, 34, \dots\}. \quad (9.35)$$

The idea is, once again, to do Hermite interpolation. To begin with, high order approximations are as usual at our disposal for the values y_0, y'_0, y_1, y'_1 by using $y_0, f(x_0, y_0), T_{kk}, f(x_0 + H, T_{kk})$, where T_{kk} is supposed to be the highest order approximation computed and used for continuation of the solution.

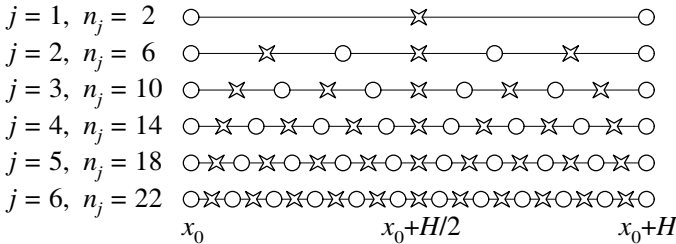


Fig. 9.7. Evaluation points for a GBS step

For more inspiration, we represent in Fig. 9.7 the steps taken by Gragg's midpoint rule for the step number sequence (9.35). The symbols $\circ$ and $\times$ indicate that the even steps and the odd steps possess a *different* asymptotic expansion (see Theorem 9.3) and must not be blended. We see that, owing to condition (9.34), the midpoint values $y_{n_j/2}^{(j)}$, obtained during the computation of T_{j1} , all have the same parity and can therefore also be extrapolated to yield an approximation for $y(x_0 + H/2)$ of order $2k - 1$ (remember that in Theorem 9.3, $\widehat{b}_{2j}(x_0) \neq 0$).

We next insert (9.20) for $x = x_0 + H/2$ into $f(x, y)$

$$f_{n_j/2}^{(j)} := f(x, y_{n_j/2}^{(j)}) = f\left(x, y(x) - h_j^2 \widehat{b}_2(x) - h_j^4 \widehat{b}_4(x) \dots\right)$$

and develop in powers of h_j to obtain

$$y'(x) - f_{n_j/2}^{(j)} = h_j^2 a_{2,1}(x) + h_j^4 a_{4,1}(x) + \dots \quad (9.36)$$

This shows that the f -values at the midpoint $x_0 + H/2$ (for $j = 1, 2, \dots, k$) possess an asymptotic expansion and can be extrapolated $k - 1$ times to yield an approximation to $y'(x_0 + H/2)$ of order $2k - 1$.

But this is not enough. We now consider, similar to an idea which goes back to the papers of Deuffhard & Nowak (1987) and Lubich (1989), the central differences $\delta f_i = f_{i+1} - f_{i-1}$ at the midpoint which, by Fig. 9.7, are available for $j = 1, 2, \dots, k$ and are based on even parity. By using (9.18) and by developing into powers of h_j we obtain

$$\begin{aligned} \frac{\delta f_{n_j/2}^{(j)}}{2h_j} &= \frac{f(x + h_j, y_{n_j/2+1}^{(j)}) - f(x - h_j, y_{n_j/2-1}^{(j)})}{2h_j} \\ &= \left(f(x + h_j, y(x + h_j) - h_j^2 \widehat{a}_2(x + h_j) - h_j^4 \widehat{a}_4(x + h_j) - \dots) - \right. \\ &\quad \left. f(x - h_j, y(x - h_j) - h_j^2 \widehat{a}_2(x - h_j) - h_j^4 \widehat{a}_4(x - h_j) - \dots) \right) / 2h_j \\ &= \frac{y'(x + h_j) - y'(x - h_j)}{2h_j} - h_j^2 c_2(x) - h_j^4 c_4(x) - \dots \end{aligned}$$

Finally we insert the Taylor series for $y'(x + h)$ and $y'(x - h)$ to obtain an expansion

$$y''(x) - \frac{\delta f_{n_j/2}^{(j)}}{2h_j} = h_j^2 a_{2,2}(x) + h_j^4 a_{4,2}(x) + \dots \quad (9.38)$$

Therefore, $k - 1$ extrapolations of the expressions (9.37) yield an approximation to $y''(x_0 + H/2)$ of order $2k - 1$.

In order to get approximations to the third and fourth derivatives of the solution at $x_0 + H/2$, we use the second and third central differences of $f_i^{(j)}$ which exist for $j \geq 2$ (Fig. 9.7). These can be extrapolated $k - 2$ times to give approximations of order $2k - 3$.

The continuation of this process yields the following algorithm:

Step 1. For each $j \in \{1, \dots, k\}$, compute approximations to the derivatives of $y(x)$ at $x_0 + H/2$ by:

$$d_j^{(0)} = y_{n_j/2}^{(j)}, \quad d_j^{(\kappa)} = \frac{\delta^{\kappa-1} f_{n_j/2}^{(j)}}{(2h_j)^{\kappa-1}} \quad \text{for } \kappa = 1, \dots, 2j. \quad (9.39)$$

Step 2. Extrapolate $d_j^{(0)}$ $(k-1)$ times and $d_j^{(2\ell-1)}$, $d_j^{(2\ell)}$ $(k-\ell)$ times to obtain improved approximations $d^{(\kappa)}$ to $y^{(\kappa)}(x_0 + H/2)$.

Step 3. For given μ ($-1 \leq \mu \leq 2k$) define the polynomial $P_\mu(\theta)$ of degree $\mu + 4$ by

$$\begin{aligned} P_\mu(0) &= y_0, & P'_\mu(0) &= Hf(x_0, y_0), \\ P_\mu(1) &= T_{kk}, & P'_\mu(1) &= Hf(x_0 + H, T_{kk}) \\ P_\mu^{(\kappa)}(1/2) &= H^\kappa d^{(\kappa)} & \text{for } \kappa = 0, \dots, \mu. \end{aligned} \quad (9.40)$$

This computation of $P_\mu(\theta)$ does not need any further function evaluation since $f(x_0 + H, T_{kk})$ has to be computed anyway for the next step. Further, $P_\mu(\theta)$ gives a global $\mathcal{C}^1$ approximation to the solution.

Theorem 9.5 (Hairer & Ostermann 1990). *If the step number sequence satisfies (9.34), then the error of the dense output polynomial $P_\mu(\theta)$ satisfies*

$$y(x_0 + \theta H) - P_\mu(\theta) = \begin{cases} \mathcal{O}(H^{2k+1}) & \text{if } n_1 = 4 \text{ and } \mu \geq 2k-4 \\ \mathcal{O}(H^{2k}) & \text{if } n_1 = 2 \text{ and } \mu \geq 2k-5. \end{cases} \quad (9.40)$$

Proof. Since $P_\mu(\theta)$ is a polynomial of degree $\mu + 4$ the error due to interpolation is of size $\mathcal{O}(H^{\mu+5})$. This explains the restriction on μ in (9.40). As explained above, the function value and derivative data used for Hermite interpolation have the required precision

$$H^\kappa y^{(\kappa)}(x_0 + H/2) - H^\kappa d^{(\kappa)} = \begin{cases} \mathcal{O}(H^{2k}) & \text{if } \kappa = 0, \\ \mathcal{O}(H^{2k+1}) & \text{if } \kappa \text{ is odd,} \\ \mathcal{O}(H^{2k+2}) & \text{if } \kappa \geq 2 \text{ is even.} \end{cases}$$

In the case $n_1 = 4$ the parity of the central point $x_0 + H/2$ is *even* (in contrary to Fig. 9.7), we therefore apply (9.18) and gain one order because then the functions $a_{i,0}(x)$ vanish at x_0 . $\square$

Control of the Interpolation Error

At one time . . . every young mathematician was familiar with $\operatorname{sn} u$, $\operatorname{cn} u$, and $\operatorname{dn} u$, and algebraic identities between these functions figured in every examination.

(E.H. Neville, *Jacobian Elliptic Functions*, 1944)

Numerical example. We apply the above dense output formula with $\mu = 2k - 3$ (as is standard in ODEX) to the differential equations of the Jacobian elliptic functions sn , cn , dn (see Abramowitz & Stegun 1964, 16.16):

$$\begin{aligned} y_1' &= y_2 y_3 & y_1(0) &= 0 \\ y_2' &= -y_1 y_3 & y_2(0) &= 1 \\ y_3' &= -0.51 \cdot y_1 y_2 & y_3(0) &= 1 \end{aligned} \quad (9.41)$$

with integration interval $0 \leq x \leq 10$ and error tolerance $Atol = Rtol = 10^{-9}$. The error for the three components of the obtained continuous solution is displayed in Fig. 9.8 (upper picture; the ghosts are the solution curves) and gives a quite disappointing impression when compared with the precision at the grid points. We shall now see that these horrible bumps are nothing else than interpolation errors.

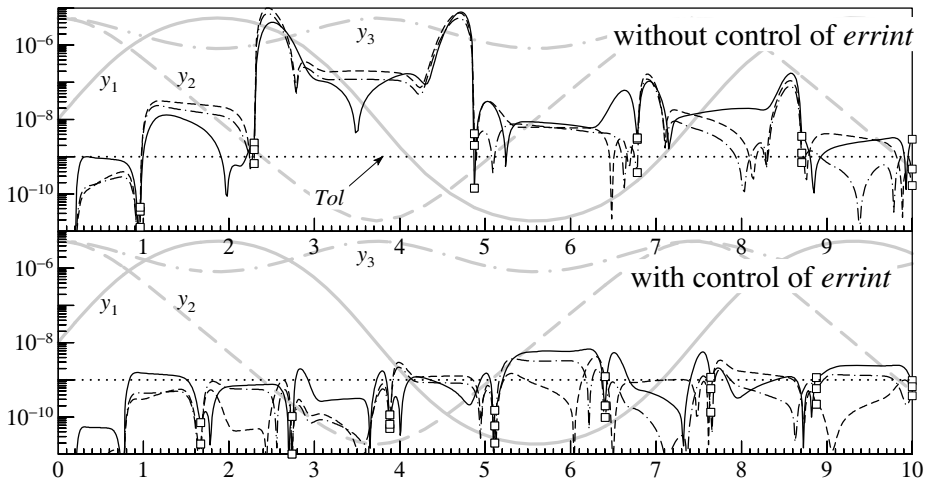


Fig. 9.8. Error of dense output without/with interpolation control

Assume that in the definition of $P_\mu(\theta)$ the basic function and derivative values are replaced by the exact values $y(x_0 + H)$, $y'(x_0 + H)$, and $y^{(\kappa)}(x_0 + H/2)$. Then the error of $P_\mu(\theta)$ is given by

$$\theta^2(1-\theta)^2 \left(\theta - \frac{1}{2} \right)^{\mu+1} \frac{y^{(\mu+5)}(\xi)}{(\mu+5)!} H^{\mu+5} \quad (9.42)$$

where $\xi \in (x_0, x_0 + H)$ (possibly different for each component). The function $\theta^2(1 - \theta)^2(\theta - 1/2)^{\mu+1}$ has its maximum at

$$\theta_{\mu+1} = \frac{1}{2} \pm \frac{1}{2} \sqrt{\frac{\mu+1}{\mu+5}} \quad (9.43)$$

which, for large μ , are close to the ends of the integration intervals and indicate precisely the locations of the large bumps in Fig. 9.8. This demonstrates the need for a code which not only controls the error at the grid points, but also takes care of the interpolation error. To this end we denote by a_μ the coefficient of $\theta^{\mu+4}$ in the polynomial $P_\mu(\theta)$ and consider (Hairer & Ostermann 1992)

$$P_\mu(\theta) - P_{\mu-1}(\theta) = \theta^2(1 - \theta)^2 \left(\theta - \frac{1}{2}\right)^\mu a_\mu \quad (9.44)$$

as an approximation for the interpolation error for $P_{\mu-1}(\theta)$ and use

$$errint = \|P_\mu(\theta_\mu) - P_{\mu-1}(\theta_\mu)\| \quad (9.45)$$

as error estimator (the norm is again that of (4.11)). Then, if $errint > 10$ the step is rejected and recomputed with

$$H_{\text{int}} = H(1/errint)^{1/(\mu+4)}$$

because $errint = \mathcal{O}(H^{\mu+4})$. Otherwise the subsequent step is computed subject to the restriction $H \leq H_{\text{int}}$.

This modified step size strategy makes the code, together with its dense output, more robust. The corresponding numerical results for the problem (9.41) are presented in the lower graph of Fig. 9.8.

Exercises

1. Show that the extrapolated Euler methods $T_{3,1}, T_{3,2}, T_{3,3}$ (with step-number sequence (9.8)) are equivalent to the Runge-Kutta methods of Table 9.1. Compute also the Runge-Kutta schemes corresponding to the first elements of the GBS algorithm.

Table 9.1. Extrapolation methods as Runge-Kutta methods

<table> <tr><td>0</td><td></td></tr> <tr><td>1/3</td><td>1/3</td></tr> <tr><td>2/3</td><td>1/3 1/3</td></tr> <tr><td></td><td>1/3 1/3 1/3</td></tr> </table>	0		1/3	1/3	2/3	1/3 1/3		1/3 1/3 1/3	<table> <tr><td>0</td><td></td></tr> <tr><td>1/2</td><td>1/2</td></tr> <tr><td>1/3</td><td>1/3 0</td></tr> <tr><td>2/3</td><td>1/3 0 1/3</td></tr> <tr><td></td><td>0 -1 1 1</td></tr> </table>	0		1/2	1/2	1/3	1/3 0	2/3	1/3 0 1/3		0 -1 1 1	<table> <tr><td>0</td><td></td></tr> <tr><td>1/2</td><td>1/2</td></tr> <tr><td>1/3</td><td>1/3 0</td></tr> <tr><td>2/3</td><td>1/3 0 1/3</td></tr> <tr><td></td><td>0 -2 3/2 3/2</td></tr> </table>	0		1/2	1/2	1/3	1/3 0	2/3	1/3 0 1/3		0 -2 3/2 3/2
0																														
1/3	1/3																													
2/3	1/3 1/3																													
	1/3 1/3 1/3																													
0																														
1/2	1/2																													
1/3	1/3 0																													
2/3	1/3 0 1/3																													
	0 -1 1 1																													
0																														
1/2	1/2																													
1/3	1/3 0																													
2/3	1/3 0 1/3																													
	0 -2 3/2 3/2																													
$T_{3,1}$ order 1	$T_{3,2}$ order 2	$T_{3,3}$ order 3																												

2. Combine (9.18) and (9.19) into the formula ($x = x_0 + kh$)

$$y(x) - y_k = \sum_{j=1}^{\ell} \left(\alpha_{2j}(x) + (-1)^k \beta_{2j}(x) \right) h^{2j} + h^{2\ell+2} E(x, h)$$

for the asymptotic expansion of the Gragg method defined by (9.13a,b).

3. (Stetter 1970). Prove that for every real b (generally between 0 and 1) the method

$$y_1 = y_0 + h \left(b f(x_0, y_0) + (1-b) f(x_1, y_1) \right)$$

$$y_{i+1} = y_{i-1} + h \left((1-b) f(x_{i-1}, y_{i-1}) + 2b f(x_i, y_i) + (1-b) f(x_{i+1}, y_{i+1}) \right)$$

possesses an expansion in powers of h^2 . Prove the same property for the smoothing step

$$S_h(x) = \frac{1}{2} \left(y_{2n} + y_{2n-1} + h(1-b) f(x_{2n-1}, y_{2n-1}) + h b f(x_{2n}, y_{2n}) \right).$$

4. (Stetter 1970). Is the Euler step (9.13a) essential for an h^2 -expansion? Prove that a first order starting procedure

$$y_1 = y_0 + h \Phi(x_0, y_0, h)$$

for (9.13a) produces an h^2 -expansion if the quantities

$y_{-1} = y_0 - h \Phi(x_0, y_0, -h)$, y_0 , and y_1 satisfy (9.13b) for $i = 0$.

5. Study the numerical instability of the extrapolation scheme for the harmonic sequence, i.e., suppose that the entries T_{11} , T_{21} , $T_{31} \dots$ are disturbed with rounding errors ε , $-\varepsilon$, $\varepsilon, \dots$ and compute the propagation of these errors into the extrapolation tableau (9.5).

Result. Due to the linearity of the extrapolation scheme, we suppose the T_{ik} equal zero and $\varepsilon = 1$. Then the results for sequence (9.8') are

1.								
-1.	-1.67							
1.	2.60	3.13						
-1.	-3.57	-5.63	-6.21					
1.	4.56	9.13	11.94	12.69				
-1.	-5.55	-13.63	-21.21	-25.35	-26.44			
1.	6.54	19.13	35.01	47.65	54.14	55.82		
-1.	-7.53	-25.63	-54.31	-84.09	-105.64	-116.30	-119.03	
1.	8.53	33.13	80.13	140.14	195.34	232.96	251.10	255.73

hence, for order 18, we lose approximately two digits due to roundoff errors.

6. (Laguerre 1883^{*}). If $a_1, a_2, \dots, a_n$ are distinct positive real numbers and $r_1, r_2, \dots, r_n$ are distinct reals, then

$$A = \begin{pmatrix} a_1^{r_1} & a_1^{r_2} & \dots & a_1^{r_n} \\ a_2^{r_1} & a_2^{r_2} & \dots & a_2^{r_n} \\ \vdots & \vdots & & \vdots \\ a_n^{r_1} & a_n^{r_2} & \dots & a_n^{r_n} \end{pmatrix}$$

is invertible.

Hint (Pólya & Szegő 1925, Vol. II, Abschn. V, Problems 76-77^{*}). Show by induction on n that, if the function $g(t) = \sum_{i=1}^n \alpha_i t^{r_i}$ has n distinct positive zeros, then $g(t) \equiv 0$. By Rolle's theorem the function

$$\frac{d}{dt}(t^{-r_1} g(t)) = \sum_{i=2}^n \alpha_i (r_i - r_1) t^{r_i - r_1 - 1}$$

has $n - 1$ positive distinct zeros and the induction hypothesis can be applied.

^{*} We are grateful to our colleague J. Steinig for these references.

II.10 Numerical Comparisons

The Pleiades seem to be among the first stars mentioned in astronomical literature, appearing in Chinese annals of 2357 B.C. . . .

(R.H. Allen, *Star names, their love and meaning*, 1899, Dover 1963)

If you enjoy fooling around making pictures, instead of typesetting ordinary text, $\text{\TeX}$ will be a source of endless frustration/amusement for you, . . .

(D. Knuth, *The $\text{\TeX}$ book*, p. 389)

Problems

EULR — Euler’s equation of rotation of a rigid body (“Diese merkwürdig symmetrischen und eleganten Formeln . . .”, A. Sommerfeld 1942, vol. I, § 26.1, Euler 1758)

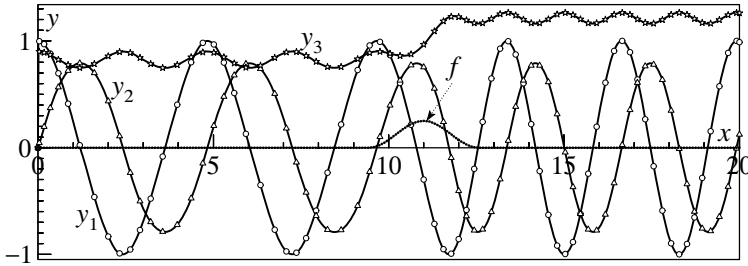


Fig. 10.1. Solutions of Euler’s equations (10.1)

$$\begin{aligned} I_1 y_1' &= (I_2 - I_3) y_2 y_3 \\ I_2 y_2' &= (I_3 - I_1) y_3 y_1 \\ I_3 y_3' &= (I_1 - I_2) y_1 y_2 + f(x) \end{aligned} \quad (10.1)$$

where y_1, y_2, y_3 are the coordinates of $\vec{\omega}$, the rotation vector, and I_1, I_2, I_3 are the principal moments of inertia. The third coordinate has an additional exterior force

$$f(x) = \begin{cases} 0.25 \cdot \sin^2 x & \text{if } 3\pi \leq x \leq 4\pi \\ 0 & \text{otherwise} \end{cases} \quad (10.1')$$

which is discontinuous in its second derivative. We choose the constants and initial values as

$$I_1 = 0.5, \quad I_2 = 2, \quad I_3 = 3, \quad y_1(0) = 1, \quad y_2(0) = 0, \quad y_3(0) = 0.9$$

(see Fig. 10.1) and check the numerical precision at the output points

$$x_{\text{end}} = 10 \quad \text{and} \quad x_{\text{end}} = 20.$$

AREN — the Arenstorf orbit (0.1) for the restricted three body problem with initial values (0.2) integrated over one period $0 \leq x \leq x_{\text{end}}$ (see Fig. 0.1). The precision is checked at the endpoint, here the solution is most sensitive to errors of the initial phase.

LRNZ — the solution of the Saltzman-Lorenz equations (I.16.17) displayed in Fig. I.16.8, i.e., with constants and initial values

$$\sigma = 10, \quad r = 28, \quad b = \frac{8}{3}, \quad y_1(0) = -8, \quad y_2(0) = 8, \quad y_3(0) = 27. \quad (10.2)$$

The solution is, for large values of x , *extremely* sensitive to the errors of the first integration steps (see Fig. I.16.10 and its discussion). For example, at $x = 50$ the numerical solution becomes totally wrong, even if the computations are performed in quadruple precision with $\text{Tot} = 10^{-20}$. Hence the numerical results of *all* methods would be equally useless and no comparison makes any sense. Therefore we choose

$$x_{\text{end}} = 16$$

and check the numerical solution at this point. Even here, all computations with $\text{Tot} \geq 10^{-7}$, say, fall into a chaotic cloud of meaningless results (see Fig. 10.5).

PLEI — a celestial mechanics problem (which we call “the Pleiades”): seven stars in the plane with coordinates x_i, y_i and masses $m_i = i$ ($i = 1, \dots, 7$):

$$\begin{aligned} x_i'' &= \sum_{j \neq i} m_j (x_j - x_i) / r_{ij} \\ y_i'' &= \sum_{j \neq i} m_j (y_j - y_i) / r_{ij} \end{aligned} \quad (10.3)$$

where

$$r_{ij} = ((x_i - x_j)^2 + (y_i - y_j)^2)^{3/2}, \quad i, j = 1, \dots, 7.$$

The initial values are

$$\begin{aligned} x_1(0) &= 3, & x_2(0) &= 3, & x_3(0) &= -1, & x_4(0) &= -3, \\ x_5(0) &= 2, & x_6(0) &= -2, & x_7(0) &= 2, \\ y_1(0) &= 3, & y_2(0) &= -3, & y_3(0) &= 2, & y_4(0) &= 0, \\ y_5(0) &= 0, & y_6(0) &= -4, & y_7(0) &= 4, \\ x_i'(0) &= y_i'(0) = 0, & \text{for all } i & \text{with the exception of} \\ x_6'(0) &= 1.75, & x_7'(0) &= -1.5, & y_4'(0) &= -1.25, & y_5'(0) &= 1, \end{aligned} \quad (10.4)$$

and we integrate for $0 \leq t \leq t_{\text{end}} = 3$. Fig. 10.2a represents the movement of these 7 bodies in phase coordinates. The initial value is marked by an “i”, the final value at $t = t_{\text{end}}$ is marked by an “f”. Between these points, 19 time-equidistant output points are plotted and connected by a dense output formula. There occur several quasi-collisions which are displayed in Table 10.1.

Table 10.1. Quasi-collisions in the PLEI problem

Body 1	1	1	3	1	2	5
Body 2	7	3	5	7	6	7
r_{ij}^2	0.0129	0.0193	0.0031	0.0011	0.1005	0.0700
time	1.23	1.46	1.63	1.68	1.94	2.14

The resulting violent shapes of the derivatives $x'_i(t), y'_i(t)$ are displayed in Fig. 10.2b and show that automatic step size control is essential for this example.

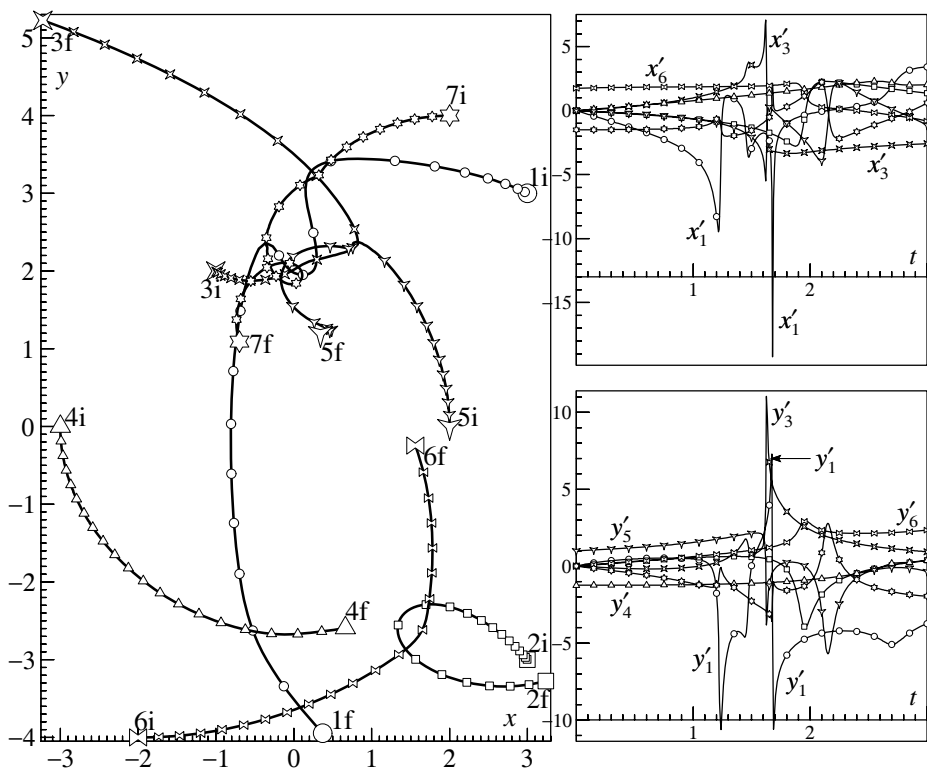


Fig. 10.2a. Solutions of (10.3)

Fig. 10.2b. Speeds

ROPE — the movement of a hanging rope (see Fig. 10.3a) of length 1 under gravitation and under the influence of a horizontal force

$$F_y(t) = \left(\frac{1}{\cosh(4t - 2.5)} \right)^4 \quad (10.5a)$$

acting at the point $s = 0.75$ as well as a vertical force

$$F_x(t) = 0.4 \quad (10.5b)$$

acting at the endpoint $s = 1$.

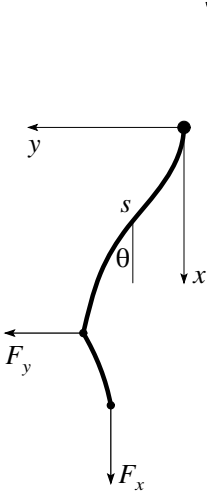


Fig. 10.3a. Hanging rope

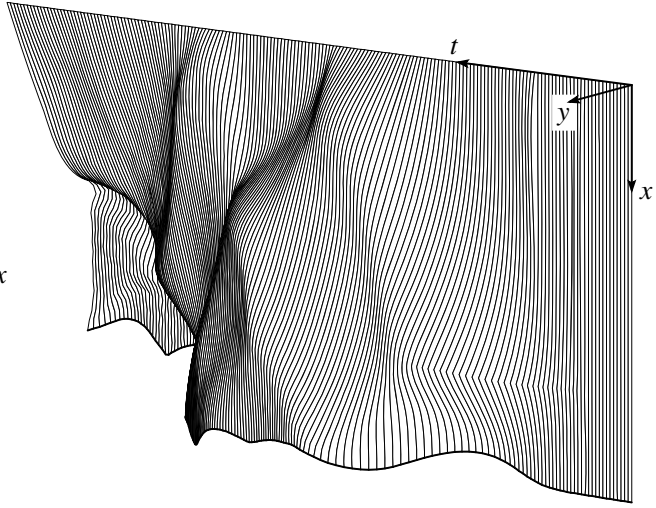


Fig. 10.3b. Solution for $0 \leq t \leq 3.723$.

If this problem is discretized, then Lagrange theory (see (I.6.18); see also Exercises IV.1.2 and IV.1.4 of Volume II) leads to the following equations for the unknown angles θ_k :

$$\sum_{k=1}^n a_{lk} \ddot{\theta}_k = - \sum_{k=1}^n b_{lk} \dot{\theta}_k^2 - n \left(n + \frac{1}{2} - l \right) \sin \theta_l - n^2 \sin \theta_l \cdot F_x(t) + \begin{cases} n^2 \cos \theta_l \cdot F_y(t) & \text{if } l \leq 3n/4 \\ 0 & \text{if } l > 3n/4, \end{cases} \quad l = 1, \dots, n \quad (10.6)$$

where

$$a_{lk} = g_{lk} \cos(\theta_l - \theta_k), \quad b_{lk} = g_{lk} \sin(\theta_l - \theta_k), \quad g_{lk} = n + \frac{1}{2} - \max(l, k). \quad (10.7)$$

We choose

$$n = 40, \quad \theta_l(0) = \dot{\theta}_l(0) = 0, \quad 0 \leq t \leq 3.723. \quad (10.8)$$

The resulting system is of dimension 80. The special structure of G^{-1} (see (IV.1.16–18) of Volume II) allows one to evaluate $\dot{\theta}_l$ with the following algorithm:

- a) Let $v_l = -n(n + \frac{1}{2} - l) \sin \theta_l - n^2 \sin \theta_l \cdot F_x + \begin{cases} n^2 \cos \theta_l \cdot F_y \\ 0 \end{cases}$
- b) Compute $w = Dv + \dot{\theta}^2$,
- c) Solve the tridiagonal system $Cu = w$,
- d) Compute $\ddot{\theta} = Cv + Du$,

where

$$C = \begin{pmatrix} 1 & -\cos(\theta_1 - \theta_2) & & & \\ -\cos(\theta_2 - \theta_1) & 2 & -\cos(\theta_2 - \theta_3) & & \\ & -\cos(\theta_3 - \theta_2) & \ddots & \ddots & \\ & & \ddots & 2 & -\cos(\theta_{n-1} - \theta_n) \\ & & & -\cos(\theta_n - \theta_{n-1}) & 3 \end{pmatrix} \quad (10.9)$$

$$D = \begin{pmatrix} 0 & -\sin(\theta_1 - \theta_2) & & & \\ -\sin(\theta_2 - \theta_1) & 0 & -\sin(\theta_2 - \theta_3) & & \\ & -\sin(\theta_3 - \theta_2) & \ddots & \ddots & \\ & & \ddots & 0 & -\sin(\theta_{n-1} - \theta_n) \\ & & & -\sin(\theta_n - \theta_{n-1}) & 0 \end{pmatrix}.$$

BRUS — the reaction-diffusion equation (Brusselator with diffusion)

$$\begin{aligned} \frac{\partial u}{\partial t} &= 1 + u^2 v - 4.4u + \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \\ \frac{\partial v}{\partial t} &= 3.4u - u^2 v + \alpha \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right) \end{aligned} \quad (10.10)$$

for $0 \leq x \leq 1$, $0 \leq y \leq 1$, $t \geq 0$, $\alpha = 2 \cdot 10^{-3}$ together with the Neumann boundary conditions

$$\frac{\partial u}{\partial \mathbf{n}} = 0, \quad \frac{\partial v}{\partial \mathbf{n}} = 0, \quad (10.11)$$

and the initial conditions

$$u(x, y, 0) = 0.5 + y, \quad v(x, y, 0) = 1 + 5x. \quad (10.12)$$

By the method of lines (cf. Section I.6) this problem becomes a system of ordinary differential equations. We put

$$x_i = \frac{i-1}{N-1}, \quad y_j = \frac{j-1}{N-1}, \quad i, j = 1, \dots, N$$

and define

$$U_{ij}(t) = u(x_i, y_j, t), \quad V_{ij}(t) = v(x_i, y_j, t). \quad (10.13)$$

Discretizing the derivatives in (10.10) with respect to the space variables we obtain for $i, j = 1, \dots, N$

$$\begin{aligned} U'_{ij} &= 1 + U_{ij}^2 V_{ij} - 4.4 U_{ij} + \alpha(N-1)^2 (U_{i+1,j} + U_{i-1,j} + U_{i,j+1} + U_{i,j-1} - 4U_{ij}) \\ V'_{ij} &= 3.4 U_{ij} - U_{ij}^2 V_{ij} + \alpha(N-1)^2 (V_{i+1,j} + V_{i-1,j} + V_{i,j+1} + V_{i,j-1} - 4V_{ij}), \end{aligned} \quad (10.14)$$

an ODE of dimension $2N^2$. Because of the boundary condition (10.11) we have

$$U_{0,j} = U_{2,j}, \quad U_{N+1,j} = U_{N-1,j}, \quad U_{i,0} = U_{i,2}, \quad U_{i,N+1} = U_{i,N-1}$$

and similarly for the V_{ij} -quantities. We choose $N = 21$ so that the system is of dimension 882 and check the numerical solutions at the output point $t_{\text{end}} = 7.5$. The solution of (10.14) (in the (x, y) -space) is represented in Fig. 10.4a and Fig. 10.4b for u and v respectively.

Performance of the Codes

Several codes were applied to each of the test problems with $Tol = 10^{-3}$, $Tol = 10^{-3-1/8}$, $Tol = 10^{-3-2/8}$, $Tol = 10^{-3-3/8}$, $\dots$ (for the large problems with $Tol = 10^{-3}$, $Tol = 10^{-3-1/4}$, $Tol = 10^{-3-2/4}$, $\dots$) up to, in general, $Tol = 10^{-14}$, then the numerical result at the output points were compared with an “exact solution” (computed very precisely in quadruple precision). Each of these results then corresponds to one point of Fig. 10.5, where this precision is compared (in double logarithmic scale) to the number of function evaluations. The “integer” tolerances 10^{-3} , 10^{-4} , 10^{-5} , $\dots$ are distinguishable as enlarged symbols. All codes were applied with complete “standard” parameter settings and were not at all “tuned” to these particular problems.

A comparison of the *computing time* (instead of the number of function evaluations) gave no significant difference. Therefore, only one representative of the small problems (LRNZ) and one large problem (BRUS) are displayed in Fig. 10.6. All computations have been performed in REAL*8 ($U_{\text{round}} = 1.11 \cdot 10^{-16}$) on a Sun Workstation (SunBlade 100).

The codes used are the following:

RKF45 — symbol $\star$ — a product of Shampine and Watts’ programming art based on Fehlberg’s pair of orders 4 and 5 (Table 5.1). The method is used in the “local extrapolation mode”, i.e., the numerical solution is advanced with the 5th order result. The code is usually, except for low precision, the slowest of all, which is explained by its low order. The results of the “time”-picture Fig. 10.6 for this

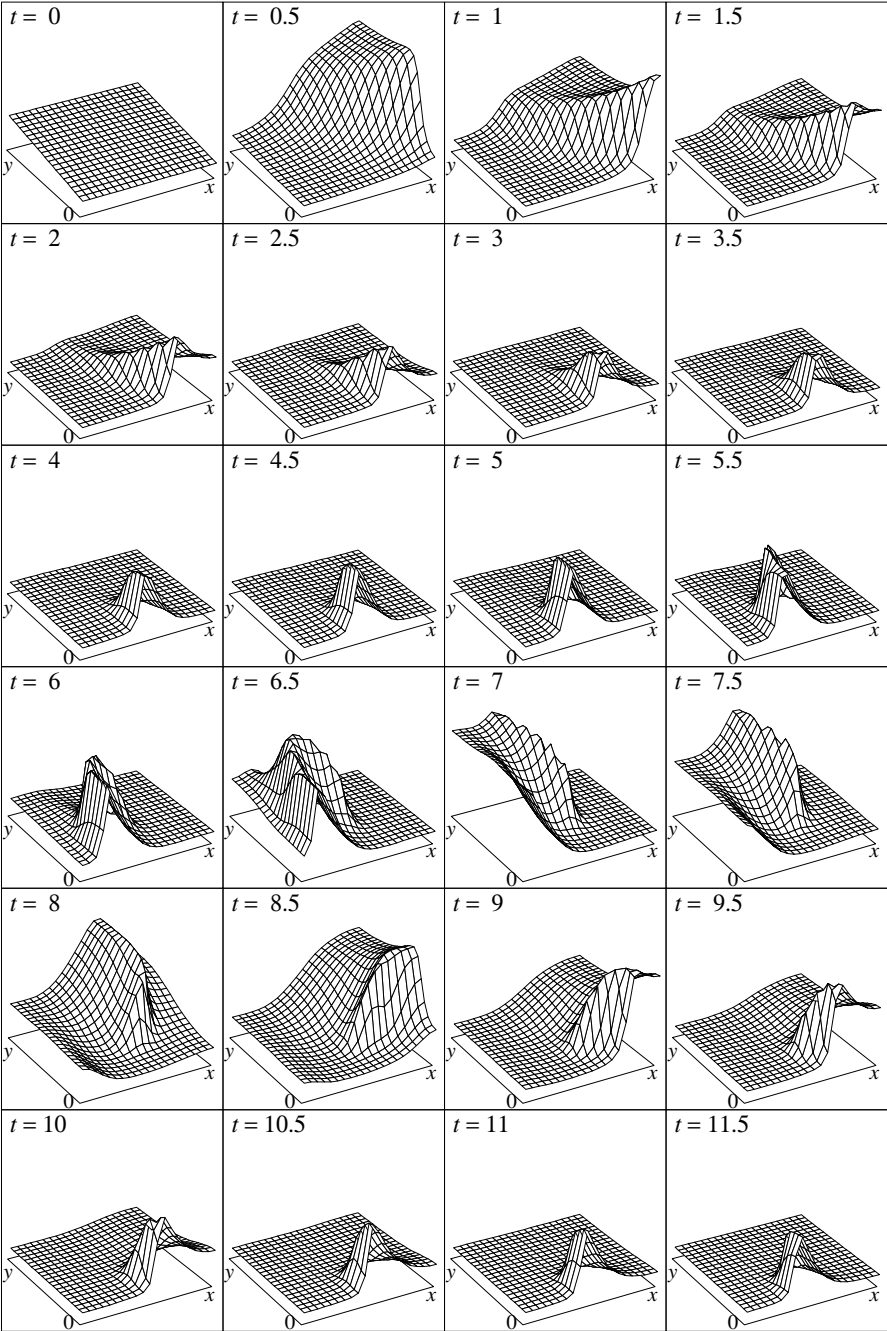


Fig. 10.4a. Solution $u(x, y, t)$ for the BRUS problem

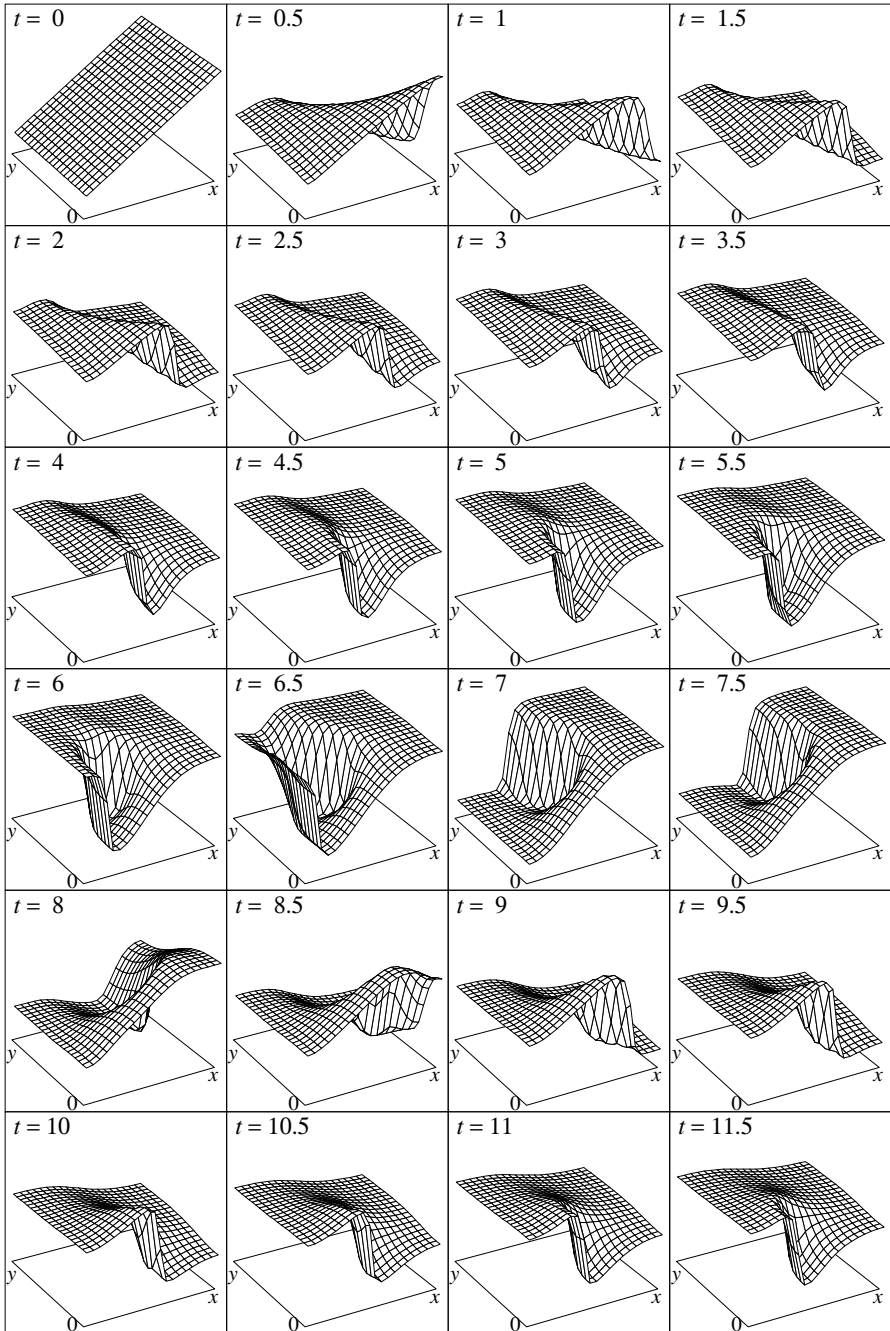


Fig. 10.4b. Solution $v(x, y, t)$ for the BRUS problem

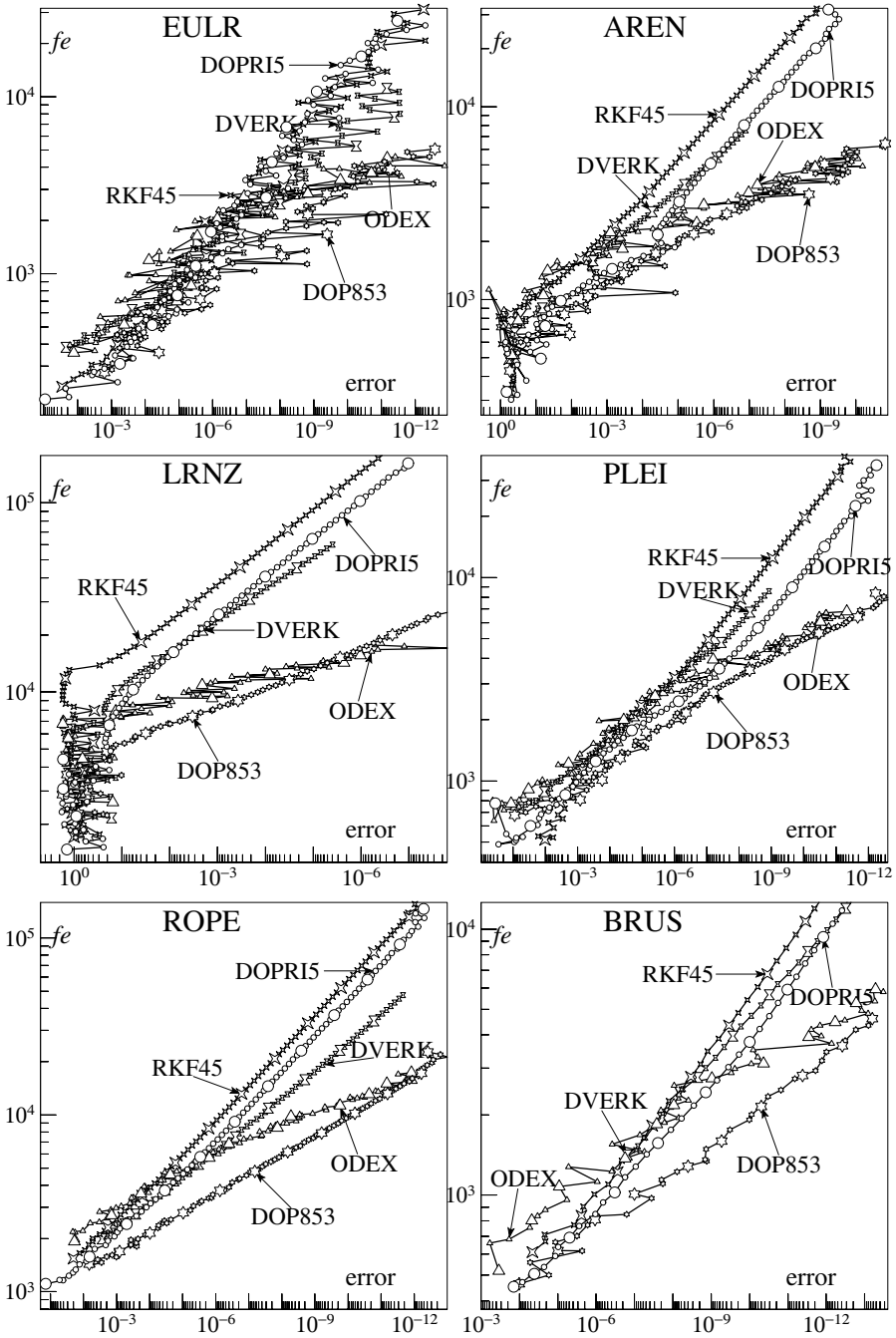


Fig. 10.5. Precision versus function calls

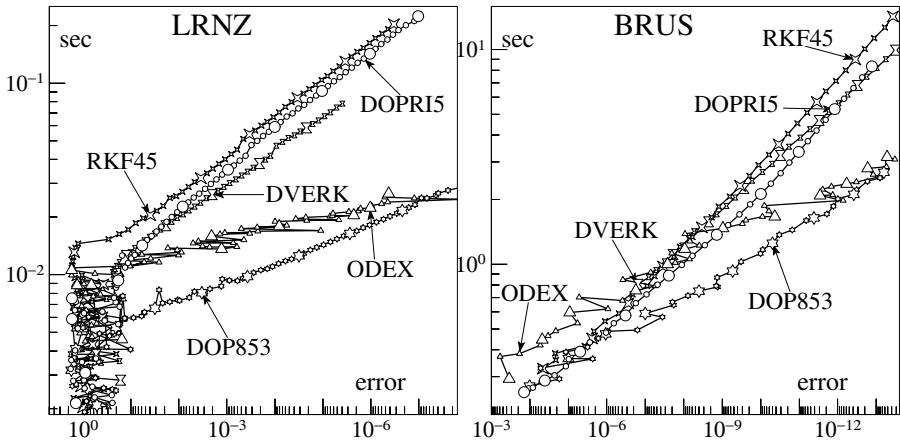


Fig. 10.6. Precision versus computing time

code are relatively better than those on the “function calls” front (Fig. 10.5). This indicates that the code has particularly small overhead.

DOPRI5 — symbol $\bigcirc$ — the method of Dormand & Prince of order 5 with embedded error estimator of order 4 (see Table 5.2). The code is explained in the Appendix. The method has precisely the same order as that used in RKF45, but the error constants are much more optimized. Therefore the “error curves” in Fig. 10.5 are nicely parallel to those of RKF45, but appear translated to the side of higher precision. One usually gains between a half and one digit of numerical precision for comparable numerical work. The code performs specially well between $Tol = 10^{-3}$ and $Tol = 10^{-8}$ in the AREN problem. This is simply due to an accidental sign change of the error for the most sensitive solution component.

DVERK — symbol Σ — this widely known code implements Verner’s 6th order method of Table 5.4 and was written by Hull, Enright & Jackson. It has been included in the IMSL library for many years and the source code is available through na-net. The corresponding error curves in Fig. 10.5 appear to be less steep than those of DOPRI5, which illustrates the higher order of the method. However, the error constants seem to be less optimal so that this code surpasses the performance of DOPRI5 only for very stringent tolerances. It is significantly better than DOPRI5 solely in problems EULR and ROPE. The code, as it was, failed at the BRUS problem for $Tol = 10^{-3}$ and $Tol = 10^{-4}$. Therefore these computations were started with $Tol = 10^{-5}$.

DOP853 — symbol $\star$ — is the method of Dormand & Prince of order 8 explained in Section II.5 (formulas (5.20) – (5.30), see Appendix). The 6th order error estimator (5.29), (5.30) has been replaced by a 5th order estimator with 3rd order correction (see below). This was necessary to make the code robust for the

EULR problem. The code works perfectly for all problems and nearly all tolerances. Whenever more than 3 or 4 digits are desired, this method seems to be highly recommendable. The most astonishing fact is that its use was never disastrous, even not for $Tol = 10^{-3}$.

ODEX — symbol $\triangle$ — is an extrapolation code based on the Gragg-Bulirsch-Stoer algorithm with harmonic step number sequence (see Appendix). This method, which allows arbitrary high orders (in the standard version of the code limited to $p \leq 18$) is of course predestined for computations with high precision. The more stringent Tol is, the higher the used order becomes, the less steep the error curve is. This can best be observed in the picture for the ROPE problem. Finally, for $Tol \approx 10^{-12}$, the code surpasses the values of DOP853. As can be seen in Fig. 10.6, the code loses slightly on the “time”-front. This is due to the increased overhead of the extrapolation scheme.

The numerical results of ODEX behave very similarly to those of DIFEX1 (Deuffhard 1983).

A “Stretched” Error Estimator for DOP853

In preliminary stages of our numerical tests we had written a code “DOPR86” based on the method of order 8 of Dormand & Prince with the 6th order error estimator described in Section II.5. For most problems the results were excellent. However, there are some situations in which the error control of DOPR86 did not work safely:

When applied to the BRUS problem with $Tol = 10^{-3}$ or $Tol = 10^{-4}$ the code stopped with an overflow message. The reason was the following: when the step size is too large, the internal stages are too far away from the solution and their modulus increases at each stage (e.g., by a factor 10^5 between stage 11 and stage 12). Due to the fact that $\hat{b}_{12} = b_{12}$ (see (5.30) (5.26) and (5.25b)) the difference $\hat{y}_1 - y_1$ is not influenced by the last stage and is smaller (by a factor of 10^5) than the modulus of y_1 . Hence, the error estimator scaled by (4.10) is $\leq 10^{-5}$ and a completely wrong step will be accepted.

The code DOPR86 also had severe difficulties when applied to problems with discontinuities such as EULR. The worst results were obtained for the problem

$$\begin{aligned} y_1' &= y_2 y_3 & y_1(0) &= 0 \\ y_2' &= -y_3 y_1 & y_2(0) &= 1 \\ y_3' &= -0.51 \cdot y_1 y_2 + f(x) & y_3(0) &= 1 \end{aligned} \quad (10.15)$$

where $f(x)$, given in (10.1'), has a discontinuous second derivative. The results for this problem and the code DOPR86 for very many different Tol values ($Tol = 10^{-3}, 10^{-3-1/24}, 10^{-3-2/24}, \dots, 10^{-14}$) are displayed in Fig. 10.7. There,

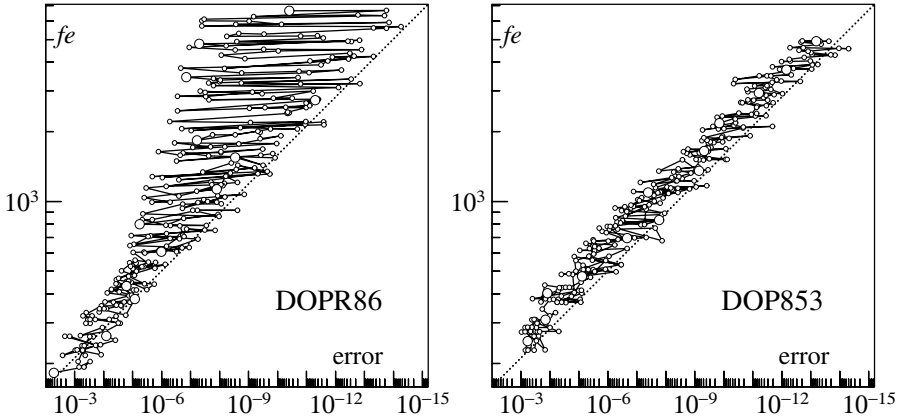


Fig. 10.7. Performances of DOPR86 and DOP853 at (10.15)

the (dotted) diagonal is of exact slope $1/8$ and represents the theoretical convergence speed of the method of order 8. It can be observed that this convergence is well attained by *some* results, but others lose precision of up to 8 digits from the desired tolerance. We explain this disappointing behaviour by the fact that $\hat{b}_{12} = b_{12}$ and that the 12th stage is the only one where the function is evaluated at the end-point of the step. Whenever the discontinuity of f'' is by accident slightly to the left of a grid point, the error estimator ignores it and the code reports a wrong value.

Unfortunately, the basic 8th order method does not possess a 6th order embedding with $\hat{b}_{12} \neq b_{12}$ (unless additional function evaluations are used). Therefore, we decided to construct a 5th order approximation $\hat{y}_1$. It can be obtained by taking $\hat{b}_6, \hat{b}_7, \hat{b}_{12}$ as free parameters, e.g.,

$$\hat{b}_6 = b_6/2 + 1, \quad \hat{b}_7 = b_7/2 + 0.45, \quad \hat{b}_{12} = b_{12}/2,$$

by putting $\hat{b}_2 = \hat{b}_3 = \hat{b}_4 = \hat{b}_5 = 0$ and by determining the remaining coefficients such that this quadrature formula has order 5. Due to the simplifying assumptions (5.20) all conditions for order 5 are then satisfied. In order to prevent a serious *over-estimation* of the error, we consider a second embedded method $\tilde{y}_1$ of order 3 based on the nodes $c_1 = 0$, c_9 and $c_{12} = 1$ so that two error estimators

$$err_5 = \|\hat{y}_1 - y_1\| = \mathcal{O}(h^6), \quad err_3 = \|\tilde{y}_1 - y_1\| = \mathcal{O}(h^4) \quad (10.16)$$

are available. Similarly to a procedure which is common for quadrature formulas (R. Piessens, E. de Doncker-Kapenga, C.W. Überhuber & D.K. Kahaner 1983, Berntsen & Espelid 1991) we consider

$$err = err_5 \cdot \frac{err_5}{\sqrt{err_5^2 + 0.01 \cdot err_3^2}} = \mathcal{O}(h^8) \quad (10.17)$$

as error estimator. It behaves asymptotically like the global error of the method. The corresponding code DOP853 gives satisfactory results for all the above problems (see right picture in Fig. 10.7).

Effect of Step-Number Sequence in ODEX

We also study the influence of the different step-number sequences to the performance of the extrapolation code ODEX. Fig. 10.8 presents two examples of this study, a small problem (AREN) and a large problem (ROPE). The used sequences are

HARMONIC — symbol $\bigcirc$ — the harmonic sequence (9.8') which is the standard choice in ODEX;

MOD4 — symbol $\triangle$ — the sequence $\{2, 6, 10, 14, 18, \dots\}$ (see (9.35)) which allowed the construction of high-order dense output;

BULIRSCH — symbol $\square$ — the Bulirsch sequence (9.7');

ROMBERG — symbol $\diamond$ — the Romberg sequence (9.6');

DNSECTRL — symbol $\star$ — the error control for the MOD4 sequence taking into account the interpolation error of the dense output solution (9.42). This is included only in the small problem, since (complete) dense output on large problems would need too much memory.

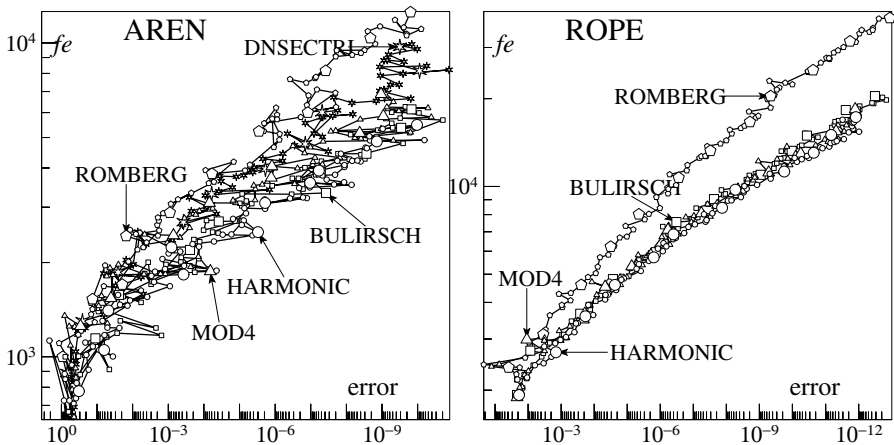


Fig. 10.8. Effect of step-number sequences in ODEX

Discussion. With the exception of the clear inferiority of the Romberg sequence, especially for high precision, and a certain price to be paid for the dense output error control, there is not much difference between the first three sequences. Although the harmonic sequence appears to be slightly superior, the difference is statistically not very significant.

II.11 Parallel Methods

We suppose that we have a computer with a number of arithmetic processors capable of simultaneous operation and seek to devise parallel integration algorithms for execution on such a computer.

(W.L. Miranker & W. Liniger 1967)

“PARALYSING ODES” (K. Burrage, talk in Helsinki 1990)

Parallel machines are computers with more than one processor and this facility might help us to speed up the computations in ordinary differential equations. This is particularly interesting for very large problems, for very costly function evaluation, or for fast real-time simulations. A second motivation is the desire to make a code, with the help of parallel computations, not necessarily faster but more robust and reliable.

Early attempts for finding parallel methods are Nievergelt (1964) and Miranker & Liniger (1967). See also the survey papers Miranker (1971) and Jackson (1991).

We distinguish today essentially between two types of parallel architectures:

SIMD (single instruction multiple data): all processors execute the same instructions with possibly different input data.

MIMD (multiple instruction multiple data): the different processors can act independently.

The exploitation of parallelism for an ordinary differential equation

$$y' = f(x, y), \quad y(x_0) = y_0 \quad (11.1)$$

can be classified into two main categories (Gear 1987, 1988):

Parallelism across the system. Often the problem itself offers more or less trivial applications for parallelism, e.g.,

- > if several solutions are required for various initial or parameter values;
- > if the right-hand side of (11.1) is very costly, but structured in such a way that the computation of *one* function evaluation can be split efficiently across the various processors;
- > space discretizations of partial differential equations (such as the Brusselator problem (10.14)) whose function evaluation can be done simultaneously for all components on an SIMD machine with thousands of processors;
- > the solution of boundary value problems with the multiple shooting method (see Section I.15) where all computations on the various sub-intervals can be done in parallel;